

5장. CSS 기초: 선택자와 박스 모델

화면상 요소 배치

학습 목표

- 외부 스타일시트로 CSS를 연결하고 관심사를 분리할 수 있습니다
- 타입·클래스·아이디·결합 선택자로 원하는 요소를 정확히 지정할 수 있습니다
- 박스 모델의 네 영역을 이해하고 box-sizing을 활용할 수 있습니다
- 명시도와 캐스케이드 규칙으로 스타일 충돌을 해결할 수 있습니다

5.1 CSS를 적용하는 세 가지 방법

도입

HTML이 뼈대라면, CSS는 색·크기·여백·배치를 결정하는 인테리어 설계도입니다. 같은 HTML에 어떻게 CSS를 연결하느냐에 따라 코드의 유지보수성이 크게 달라집니다. 이 절에서는 세 가지 방법을 비교하고, 실무에서 권장되는 외부 스타일시트 방식을 중심으로 살펴봅니다.

핵심 개념 설명

인라인·내부·외부 스타일의 차이

CSS를 HTML 문서에 적용하는 방법은 세 가지입니다.

인라인 스타일(Inline Style)은 태그의 `style` 속성에 직접 씁니다. 빠르게 테스트할 때 편리하지만, 재사용이 불가능하고 유지보수가 어렵습니다.

내부 스타일시트(Internal Stylesheet)는 `<head>` 안에 `<style>` 태그를 두어 규칙을 작성합니다. 같은 파일에 HTML과 CSS가 섞여 있어 여러 페이지에서 공유할 수 없습니다.

공유 안 해야 될 때 쓸 수도 있음.(단점이라고만 볼 수는 없음.)

외부 스타일시트(External Stylesheet)는 별도 `.css` 파일에 모든 규칙을 작성하고, `<link>` 태그로 연결합니다. HTML과 CSS의 **관심사 분리(Separation of Concerns)**를 실현하는 가장 권장되는 방식입니다.

베스트 프랙티스: 실무에서는 항상 외부 스타일시트를 사용합니다. 인라인 스타일은 JavaScript로 동적 제어할 때나 테스트 목적 외에는 지양합니다.

코드로 확인하기

세 가지 방법을 한 파일에서 비교합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS 적용 방법 비교</title>

  <!-- 방법 2: 내부 스타일시트 -->
  <style>
    .internal {
      color: steelblue;
      font-weight: bold;
    }
  </style>

  <!-- 방법 3: 외부 스타일시트 (별도 style.css 파일 필요) -->
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- 방법 1: 인라인 스타일 -->
  <p style="color: tomato; font-size: 18px;">인라인 스타일로 적용한 문단입니다.</p>

  <!-- 방법 2: 내부 스타일시트 클래스 사용 -->
  <p class="internal">내부 스타일시트로 적용한 문단입니다.</p>

  <!-- 방법 3: 외부 스타일시트 클래스 사용 -->
  <p class="external">외부 스타일시트로 적용한 문단입니다.</p>
</body>
</html>
```

```
/* style.css */
.external {
  color: seagreen;
  font-style: italic;
}
```

브라우저 렌더링 결과:

인라인 스타일로 적용한 문단입니다. ← 빨간색(tomato), 크기 18px
 내부 스타일시트로 적용한 문단입니다. ← 파란색(steelblue), 굵은 글씨
 외부 스타일시트로 적용한 문단입니다. ← 초록색(seagreen), 기울임꼴

줄별 코드 설명

줄	코드	설명
7~11	<code><style> ... </style></code>	내부 스타일시트. head 안에 위치하며 해당 HTML 파일에서만 유효합니다
14	<code><link rel="stylesheet" href="s...</code>	외부 CSS 파일을 연결합니다. <code>rel="stylesheet"</code> 는 연결 관계를 명시하고 <code>href</code> 에 파일 경로를 씁니다
17	<code>style="color: tomato; font-siz...</code>	인라인 스타일. 해당 요소에만 적용되며 세미콜론으로 여러 선언을 구분합니다
20	<code>class="internal"</code>	내부 스타일시트에서 정의한 <code>.internal</code> 클래스를 적용합니다
23	<code>class="external"</code>	외부 CSS에서 정의한 <code>.external</code> 클래스를 적용합니다

link 태그로 외부 CSS 연결하기

```
<link rel="stylesheet" href="경로">
```

항목	내용
설명	외부 CSS 파일을 현재 HTML 문서에 연결합니다
속성	<code>rel</code> (필수): 링크 관계 유형. CSS 파일에는 항상 <code>"stylesheet"</code> 를 씁니다
	<code>href</code> (필수): CSS 파일의 경로(상대 경로 또는 절대 URL)
	<code>type</code> : <code>"text/css"</code> (HTML5에서는 생략 가능)
위치	<code><head></code> 내부에 배치합니다
사용 예시	<code><link rel="stylesheet" href="css/style.css"></code>

외부 스타일시트를 권장하는 이유

- **재사용성**: 여러 HTML 파일이 같은 CSS 파일을 공유할 수 있습니다.
- **캐싱 효율**: 브라우저가 CSS 파일을 캐시하면 다음 방문 시 재다운로드하지 않습니다.
- **유지보수**: 디자인을 바꿀 때 CSS 파일 하나만 수정하면 모든 페이지에 반영됩니다.
- **관심사 분리**: HTML(구조)과 CSS(표현)가 명확히 분리됩니다.

비교 테이블

방법	작성 위치	적용 범위	재사용	권장 여부
인라인 스타일	태그의 <code>style</code> 속성	해당 요소 1개	불가	지양
내부 스타일시트	<code><head></code> 내 <code><style></code>	해당 HTML 파일 전체	불가	간단한 예제만
외부 스타일시트	별도 <code>.css</code> 파일	연결한 모든 파일	가능	권장

절 요약

- CSS를 HTML에 연결하는 방법은 인라인·내부·외부 스타일 세 가지입니다.
- 외부 스타일시트는 `<link rel="stylesheet" href="...">` 태그로 연결하며 가장 권장됩니다.
- 관심사 분리 원칙에 따라 구조(HTML)와 표현(CSS)을 파일로 나누면 유지보수가 쉬워집니다.

5.2 선택자로 요소 지정하기

도입

스타일을 적용하려면 먼저 "**누구에게 적용할지**"를 CSS에 알려야 합니다. 이때 사용하는 것이 **선택자(Selector)**입니다. 선택자를 교실에 비유하면, '3반 전원'처럼 무리를 지목하거나 '반장'처럼 단 한 명을 특정할 수 있습니다. 이 절에서는 자주 쓰이는 선택자 유형과 결합 방법을 학습합니다.

핵심 개념 설명

CSS 선택자 (CSS Selectors)

CSS 규칙은 **선택자(Selector)**와 **선언 블록(Declaration Block)**으로 구성됩니다.

```
/* 선택자 { 속성: 값; } */  
h1 { color: navy; }
```

선택자의 종류를 하나씩 살펴봅니다.

코드로 확인하기 — 타입·클래스·아이디 선택자

Tag	속성	속성
-----	----	----

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>기본 선택자</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h2>타입 선택자로 스타일 적용</h2>
  <p>이 문단은 타입 선택자(p)로 색이 지정됩니다.</p>

  <p class="highlight">클래스 선택자(.highlight)로 강조된 문단입니다.</p>
  <p class="highlight">같은 클래스는 여러 요소에 쓸 수 있습니다.</p>

  <p id="intro">아이디 선택자(#intro)로 지정된 특별한 문단입니다.</p>
</body>
</html>

```

```

/* style.css */

/* 타입 선택자: 모든 <p> 요소에 적용 */
p {
  color: #333;
  line-height: 1.6;
}

/* 클래스 선택자: class="highlight"인 요소에 적용 */
.highlight {
  background-color: #fff9c4;
  padding: 4px 8px;
  border-left: 4px solid #f4c430;
}

/* 아이디 선택자: id="intro"인 요소에 적용 */
#intro {
  font-size: 1.1rem;
  color: #1a73e8;
  font-style: italic;
}

```

브라우저 렌더링 결과:

타입 선택자로 스타일 적용 ← h2 기본 스타일
 이 문단은 타입 선택자(p)로 색이... ← 어두운 회색(#333)
 클래스 선택자(.highlight)로 강조... ← 노란 배경 + 왼쪽 황금색 테두리
 같은 클래스는 여러 요소에 쓸 수... ← 동일한 노란 배경 스타일
 아이디 선택자(#intro)로 지정된... ← 파란색, 기울임꼴

줄별 코드 설명

줄	코드	설명
3~6	<code>p { color: #333; ... }</code>	타입 선택자. 페이지의 모든 <code><p></code> 요소에 적용됩니다
9~13	<code>.highlight { ... }</code>	클래스 선택자. 점(.)으로 시작하며 <code>class="highlight"</code> 인 모든 요소에 적용됩니다
16~20	<code>#intro { ... }</code>	아이디 선택자. 샵(#)으로 시작하며 <code>id="intro"</code> 인 요소에 적용됩니다. 페이지에서 아이디는 유일해야 합니다

팁: 클래스 선택자(.)는 한 페이지에서 여러 요소에 반복 사용할 수 있어 재사용성이 높습니다. 아이디 선택자(#)는 페이지 내 유일한 요소에만 사용합니다.

주의: 아이디 선택자를 자주 쓰면 명시도가 매우 높아져 나중에 스타일을 재정의하기 어려워집니다. 레이아웃 구분이나 JavaScript 접근에 아이디를 쓰고, 스타일에는 클래스를 우선합니다.

코드로 확인하기 — 자손·자식·인접 형제 결합자

결합자(Combinator)는 두 선택자의 관계를 정의합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>결합 선택자</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav>
    <a href="#">메뉴 1</a>
    <a href="#">메뉴 2</a>
  </nav>

  <section class="card">
    <h3>카드 제목</h3>
    <p>카드 안의 문단입니다.</p>
    <div>
      <p>div 안의 문단(자손)입니다.</p>
    </div>
  </section>

  <h3>섹션 밖 제목</h3>
  <p>첫 번째 인접 문단 (h3 바로 다음)</p>
  <p>두 번째 문단 (인접하지 않음)</p>
</body>
</html>
```



```

/* style.css */

/* 자손 결합자(공백): .card 안의 모든 p (깊이 무관) */
.card p {
  color: #555;
  margin: 8px 0;
}

/* 자식 결합자(>): .card의 직접 자식인 p만 */
.card > p {
  font-weight: bold;
}

/* 인접 형제 결합자(+): h3 바로 다음에 오는 p 하나 */
h3 + p {
  color: tomato;
  font-style: italic;
}

/* 그룹 선택자(,): 여러 선택자에 같은 스타일 적용 */
nav a, .card h3 {
  text-decoration: none;
  color: #1a73e8;
}

```

브라우저 렌더링 결과:

메뉴1 메뉴2	← nav a: 파란색 링크, 밑줄 없음
카드 제목	← .card h3: 파란색
카드 안의 문단입니다.	← .card > p: 굵은 글씨 + 회색
div 안의 문단(자손)입니다.	← .card p: 회색 (굵지 않음)
섹션 밖 제목	
첫 번째 인접 문단 (h3 바로 다음)	← h3 + p: 빨간색 기울임꼴
두 번째 문단 (인접하지 않음)	← 기본 스타일

줄별 코드 설명

줄	코드	설명
3~6	<code>.card p { ... }</code>	자손 결합자. <code>.card</code> 내부에 있는 모든 <code>p</code> 에 적용됩니다 (중첩 깊이 무관)
9~11	<code>.card > p { ... }</code>	자식 결합자. <code>.card</code> 의 직접 자식 <code>p</code> 에만 적용됩니다
14~17	<code>h3 + p { ... }</code>	인접 형제 결합자. <code>h3</code> 바로 다음에 오는 하나의 <code>p</code> 에만 적용됩니다
20~23	<code>nav a, .card h3 { ... }</code>	그룹 선택자. 쉼표로 구분하여 여러 선택자에 같은 규칙을 적용합니다

코드로 확인하기 — 속성 선택자

속성 선택자(Attribute Selector)는 HTML 속성의 존재나 값을 조건으로 요소를 선택합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>속성 선택자</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <input type="text" placeholder="텍스트 입력">
  <input type="email" placeholder="이메일 입력">
  <input type="password" placeholder="비밀번호 입력">
  <a href="https://example.com">외부 링크</a>
  <a href="/about">내부 링크</a>
</body>
</html>
```

```

/* style.css */

/* type 속성이 "email"인 input */
input[type="email"] {
  border: 2px solid #1a73e8;
  border-radius: 4px;
}

/* href 속성이 "https://"로 시작하는 a */
a[href^="https://"] {
  color: green;
}
a[href^="https://"]::after {
  content: " 🔒";
}

```

브라우저 렌더링 결과:

[텍스트 입력] ← 기본 테두리
 [이메일 입력] ← 파란색 테두리 (type="email")
 [비밀번호 입력] ← 기본 테두리
 외부 링크 🔒 ← 초록색 + 외부 표시 아이콘
 내부 링크 ← 기본 링크 색

줄별 코드 설명

줄	코드	설명
3~6	<code>input[type="email"] { ... }</code>	<code>type</code> 속성 값이 정확히 <code>"email"</code> 인 input 요소를 선택합니다
9~11	<code>a[href^="https://"]</code>	<code>href</code> 속성이 <code>"https://"</code> 로 시작하는 a 요소를 선택합니다. <code>^=</code> 는 "시작" 조건입니다
12~14	<code>::after { content: " 🔒"; }</code>	외부 링크 뒤에 화살표 기호를 추가합니다 (의사 요소)

비교 테이블

선택자 종류	문법 예시	선택 대상
타입 선택자	<code>p</code> , <code>h1</code>	해당 태그 이름의 모든 요소
클래스 선택자	<code>.card</code>	<code>class="card"</code> 요소 전체
아이디 선택자	<code>#header</code>	<code>id="header"</code> 요소 1개
자손 결합자	<code>.wrap p</code>	<code>.wrap</code> 안 모든 <code>p</code>
자식 결합자	<code>.wrap > p</code>	<code>.wrap</code> 의 직접 자식 <code>p</code>
인접 형제	<code>h2 + p</code>	<code>h2</code> 바로 다음 <code>p</code>
그룹 선택자	<code>h1, h2</code>	<code>h1</code> 과 <code>h2</code> 모두
속성 선택자	<code>input[type="text"]</code>	<code>type="text"</code> 인 input

절 요약

- 타입 선택자는 태그 이름으로, 클래스 선택자는 `.` 으로, 아이디 선택자는 `#` 으로 요소를 선택합니다.
- 결합자(공백, `>`, `+`)를 사용해 요소 간 관계를 조건으로 선택할 수 있습니다.
- 속성 선택자(`[attr="값"]`)로 속성 조건에 맞는 요소만 선택할 수 있습니다.
- 스타일 재사용성을 위해 클래스 선택자를 우선 사용합니다.

5.3 박스 모델 이해하기

도입

CSS에서 가장 중요한 개념 중 하나가 **박스 모델(Box Model)**입니다. 브라우저는 모든 HTML 요소를 사각형 상자로 인식합니다. 선물 상자에 비유하면, 선물 자체(content), 완충재(padding), 상자 벽(border), 다른 상자와의 간격(margin)으로 구성됩니다. 박스 모델을 이해해야 여백과 크기를 정확히 제어할 수 있습니다.

핵심 개념 설명

박스 모델 (Box Model)

모든 요소는 안쪽에서 바깥쪽으로 다음 네 영역을 갖습니다.

- **콘텐츠(Content)**: 실제 내용(텍스트, 이미지)이 위치하는 영역. `width` · `height` 로 크기를 지정합니다.
- **패딩(Padding)**: 콘텐츠와 테두리 사이의 안쪽 여백. 배경색이 패딩까지 채워집니다.
- **테두리(Border)**: 요소를 둘러싸는 선. `border-width` · `border-style` · `border-color` 로 제어합니다.
- **마진(Margin)**: 요소 바깥의 여백. 다른 요소와의 간격을 만듭니다.

CSS 박스 모델 (Box Model) 동심 사각형 다이어그램

코드로 확인하기 — 박스 모델 기본

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>박스 모델 기본</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="box">박스 모델 예시</div>
  <div class="box-border-box">border-box 예시</div>
</body>
</html>
```

```

/* style.css */

/* 기본 box-sizing: content-box (브라우저 기본값) */
.box {
  width: 200px;
  height: 80px;
  padding: 20px;
  border: 4px solid #333;
  margin: 30px;
  background-color: #e3f2fd;
  /* 실제 렌더링 너비: 200 + 20*2 + 4*2 = 248px */
}

/* box-sizing: border-box 적용 */
.box-border-box {
  box-sizing: border-box;
  width: 200px;
  height: 80px;
  padding: 20px;
  border: 4px solid #388e3c;
  margin: 30px;
  background-color: #e8f5e9;
  /* 실제 렌더링 너비: 200px (padding과 border가 내부에 포함) */
}

```

브라우저 렌더링 결과:

[박스 모델 예시]	← 실제 너비 248px (content-box)
[border-box 예시]	← 실제 너비 200px (border-box)

줄별 코드 설명

줄	코드	설명
4	<code>width: 200px;</code>	콘텐츠 영역의 너비를 200px로 지정합니다. <code>content-box</code> 에서 padding-border는 별도로 추가됩니다
5	<code>height: 80px;</code>	콘텐츠 영역의 높이를 80px로 지정합니다
6	<code>padding: 20px;</code>	네 방향 안쪽 여백을 모두 20px로 지정합니다 (상하좌우 동일)
7	<code>border: 4px solid #333;</code>	4px 두께의 실선 테두리를 지정합니다
8	<code>margin: 30px;</code>	네 방향 바깥쪽 여백을 30px로 지정합니다
16	<code>box-sizing: border-box;</code>	padding과 border를 width 안에 포함시킵니다. 지정한 너비가 실제 렌더링 너비가 됩니다

width와 height가 적용되는 범위

기본값인 `box-sizing: content-box` 에서 `width` 는 콘텐츠 영역만의 너비입니다. 실제 렌더링 너비는 다음과 같이 계산됩니다.

```

실제 너비 = width + padding-left + padding-right + border-left + border-right
           = 200   + 20           + 20           + 4           + 4
           = 248px

```

이 계산 방식은 직관적이지 않아 레이아웃을 맞추기 어렵게 만듭니다.

box-sizing: border-box로 계산 단순화하기

`box-sizing: border-box` 를 설정하면 `width` 안에 padding과 border가 포함됩니다. 지정한 `width: 200px` 이 실제 렌더링 너비 200px이 됩니다.

모던 CSS에서는 * 선택자로 전체 요소에 적용하는 방법을 널리 사용합니다.

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

베스트 프랙티스: 모든 프로젝트 CSS 파일의 첫 줄에 위 `box-sizing: border-box` 초기화 코드를 작성하면 크기 계산이 훨씬 단순해집니다.

코드로 확인하기 — 패딩과 마진 단축 표기

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>패딩과 마진 단축 표기</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="card">카드 컴포넌트</div>
</body>
</html>
```



```

/* style.css */
.card {
  box-sizing: border-box;
  width: 300px;
  background-color: #fff;
  border: 1px solid #ddd;
  border-radius: 8px;

  /* 단축 표기: 상하 16px, 좌우 24px */
  padding: 16px 24px;

  /* 단축 표기: 상 0, 우 auto, 하 32px, 좌 auto (가운데 정렬) */
  margin: 0 auto 32px;
}

```

줄별 코드 설명

줄	코드	설명
9	<code>padding: 16px 24px;</code>	두 값일 때: 첫 번째는 상하, 두 번째는 좌우입니다
12	<code>margin: 0 auto 32px;</code>	세 값일 때: 상(0), 좌우(auto), 하(32px). <code>auto</code> 는 남은 공간을 자동 분배합니다

마진 상쇄(margin collapsing) 현상

세로 방향(위·아래)의 마진이 만나면 큰 값 하나로 합쳐지는 현상입니다.

```

.first { margin-bottom: 20px; }
.second { margin-top: 30px; }
/* 두 요소 사이 간격: 50px가 아니라 30px (큰 값 하나) */

```

주의: 마진 상쇄는 수직 방향에서만 발생하며, 가로 방향 마진은 합쳐지지 않습니다.

`display: flex` 컨테이너 안에서는 마진 상쇄가 발생하지 않습니다.

절 요약

- 모든 HTML 요소는 content·padding·border·margin으로 구성된 박스 모델을 가집니다.
 - 기본값 `content-box` 에서 `width` 는 콘텐츠 영역만의 너비이며 패딩·테두리가 추가됩니다.
 - `box-sizing: border-box` 를 사용하면 지정한 `width` 가 실제 렌더링 너비와 일치합니다.
 - 수직 방향 마진은 상쇄(collapsing)가 발생하므로 주의합니다.
-

5.4 색상, 단위, 글꼴

도입

색상과 단위는 웹 디자인의 기본 재료입니다. 색을 표현하는 방법과 크기를 지정하는 단위를 정확히 알아야 일관된 디자인을 만들 수 있습니다. 특히 `em` 과 `rem` 같은 상대 단위는 접근성 높은 웹 페이지를 만드는 핵심입니다.

핵심 개념 설명

색상 표기 방법: hex·rgb·hsl

CSS에서 색상을 지정하는 세 가지 주요 방법이 있습니다.

- **hex(16진수):** `#RRGGBB` 또는 `#RGB` 형식. 가장 많이 쓰이는 표기법입니다.
- **rgb():** 빨강(Red)·초록(Green)·파랑(Blue) 각각을 0~255 값으로 지정합니다.
- **hsl():** 색조(Hue, 0~360°)·채도(Saturation, %)·명도(Lightness, %)로 지정합니다. 직관적으로 색을 조절할 때 유리합니다.

코드로 확인하기 — 색상 표기

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>색상 표기 비교</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="swatch hex">hex: #3498db</div>
  <div class="swatch rgb">rgb(52, 152, 219)</div>
  <div class="swatch hsl">hsl(204, 70%, 53%)</div>
  <div class="swatch alpha">rgba(52, 152, 219, 0.5) - 반투명</div>
</body>
</html>

```

```

/* style.css */
.swatch {
  padding: 16px 24px;
  margin: 8px 0;
  color: white;
  font-weight: bold;
  border-radius: 6px;
}

.hex { background-color: #3498db; }
.rgb { background-color: rgb(52, 152, 219); }
.hsl { background-color: hsl(204, 70%, 53%); }
.alpha { background-color: rgba(52, 152, 219, 0.5); color: #333; }

```

브라우저 렌더링 결과:

```

[hex: #3498db           ] ← 파란색 박스
[rgb(52, 152, 219)      ] ← 동일한 파란색 박스
[hsl(204, 70%, 53%)     ] ← 거의 동일한 파란색 박스
[rgba(52, 152, 219, 0.5)] ← 반투명 파란색 박스

```

줄별 코드 설명

줄	코드	설명
9	<code>background-color: #3498db;</code>	16진수 표기. <code>34</code> =빨강, <code>98</code> =초록, <code>db</code> =파랑
10	<code>background-color: rgb(52, 152, ...</code>	10진수 RGB. 각 채널을 0~255 범위로 지정합니다
11	<code>background-color: hsl(204, 70%...</code>	HSL 표기. 204°=파란 계열 색조, 70%=적당한 채도, 53%=중간 명도
12	<code>background-color: rgba(52, 152...</code>	<code>rgba</code> 의 네 번째 값은 투명도(alpha). 0(완전 투명)~1(완전 불투명) 범위입니다

절대 단위와 상대 단위

CSS 단위는 크게 절대 단위와 상대 단위로 나뉩니다.

절대 단위(Absolute Unit): - `px` (픽셀): 화면 픽셀 하나에 해당하는 고정 크기. 정밀한 크기 지정에 적합합니다.

상대 단위(Relative Unit): - `em`: 부모 요소의 `font-size`를 1로 삼는 상대 크기. `2em`이면 부모의 2배입니다. - `rem`: 루트 요소(`html`)의 `font-size`를 기준으로 하는 상대 크기. 브라우저 기본값은 `16px`이므로 `1rem = 16px`입니다. - `%`: 부모 요소의 크기에 대한 비율입니다.

팁: 접근성을 위해 글자 크기는 `rem` 단위를 사용하는 것이 좋습니다. 사용자가 브라우저 기본 글자 크기를 늘려도 `rem` 기반 레이아웃은 함께 커집니다.

코드로 확인하기 — 단위 비교

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>단위 비교</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <section class="parent">
    <p class="px-text">16px - 고정 크기</p>
    <p class="em-text">1.5em - 부모(20px) 기준 30px</p>
    <p class="rem-text">1.5rem - 루트(16px) 기준 24px</p>
    <div class="percent-box">부모 너비의 50%</div>
  </section>
</body>
</html>

```

```

/* style.css */
html {
  font-size: 16px; /* 루트 기본값 명시 */
}

.parent {
  font-size: 20px; /* em 기준이 될 부모 폰트 크기 */
  width: 400px;
  background-color: #f5f5f5;
  padding: 16px;
}

.px-text { font-size: 16px; color: #333; }
.em-text { font-size: 1.5em; color: steelblue; } /* 20px * 1.5 = 30px */
.rem-text { font-size: 1.5rem; color: seagreen; } /* 16px * 1.5 = 24px */

.percent-box {
  width: 50%; /* 부모 400px의 50% = 200px */
  height: 40px;
  background-color: #ffe0b2;
}

```

브라우저 렌더링 결과:

16px - 고정 크기 ← 16px 텍스트
1.5em - 부모(20px) 기준 30px ← 30px 텍스트 (em은 부모 기준)
1.5rem - 루트(16px) 기준 24px ← 24px 텍스트 (rem은 루트 기준)
[부모 너비의 50%] ← 너비 200px 박스

줄별 코드 설명

줄	코드	설명
2	<code>html { font-size: 16px; }</code>	루트 폰트 크기를 명시합니다. rem 계산의 기준이 됩니다
7	<code>.parent { font-size: 20px; }</code>	em 계산의 기준이 될 부모 폰트 크기를 20px로 지정합니다
14	<code>font-size: 1.5em;</code>	부모 폰트 크기(20px) × 1.5 = 30px입니다
15	<code>font-size: 1.5rem;</code>	루트 폰트 크기(16px) × 1.5 = 24px입니다. 중첩 계층과 무관하게 일정합니다
18	<code>width: 50%;</code>	부모 요소 너비(400px)의 50% = 200px입니다

글꼴 지정과 웹 폰트 불러오기

기본 글꼴은 `font-family` 속성으로 지정하며, 여러 폰트를 쉼표로 나열하면 앞에서부터 사용 가능한 폰트를 찾습니다. 마지막으로 `serif`, `sans-serif` 같은 범용 계열명을 씁니다.

Google Fonts를 통해 웹 폰트를 쉽게 적용할 수 있습니다.

```

<!-- head에 Google Fonts link 추가 -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Noto+Sans+KR:wght@400;700

```

```
body {  
  font-family: 'Noto Sans KR', sans-serif;  
}
```

팁: 웹 폰트를 불러올 때 `rel="preconnect"` 를 추가하면 DNS 연결을 미리 설정하여 폰트 로딩 속도가 빨라집니다.

비교 테이블 — 주요 단위

단위	종류	기준	주요 용도
px	절대	화면 픽셀	정밀한 크기, 테두리 굵기
em	상대	부모 font-size	여백(padding/margin)에 적합
rem	상대	루트(html) font-size	글자 크기, 반응형 레이아웃
%	상대	부모 요소 크기	너비, 유동 레이아웃
vw	상대	뷰포트 너비	전체 화면 기준 크기 (ch_07)
vh	상대	뷰포트 높이	전체 화면 높이 기준 (ch_07)

절 요약

- 색상은 `#hex`, `rgb()`, `hsl()` 세 방법으로 표기하며, 투명도는 `rgba()` 와 `hsla()` 를 사용합니다.
- `px` 는 고정 크기, `em` 은 부모 기준, `rem` 은 루트 기준 상대 단위입니다.
- 글자 크기에는 접근성을 고려해 `rem` 단위를 권장합니다.
- Google Fonts 같은 웹 폰트 서비스로 다양한 폰트를 손쉽게 적용할 수 있습니다.

5.5 우선순위와 명시도

도입

같은 요소에 여러 CSS 규칙이 충돌한다면 브라우저는 어느 규칙을 따를까요? CSS는 이를 결정하는 명확한 알고리즘을 가지고 있습니다. 이 알고리즘을 이해하면 스타일이 왜 예상대로 적용되지 않는지 진단하고 해결할 수 있습니다.

핵심 개념 설명

캐스케이드(Cascade)와 명시도(Specificity)

CSS는 **캐스케이드(Cascade, 폭포)** 방식으로 동작합니다. 여러 규칙이 충돌할 때 다음 순서로 우선순위를 결정합니다.

- 1. **중요도(Importance)**: `!important` 선언이 있으면 우선합니다.
- 2. **명시도(Specificity)**: 선택자의 구체성 점수가 높을수록 우선합니다.
- 3. **선언 순서(Source Order)**: 명시도가 같다면 나중에 선언된 규칙이 우선합니다.

명시도 계산 규칙은 네 자리 점수로 표현합니다.

선택자 종류	점수	예시
인라인 스타일	1,0,0,0	<code>style=" ... "</code>
아이디 선택자	0,1,0,0	<code>#header</code>
클래스·속성·의사 클래스	0,0,1,0	<code>.btn</code> , <code>[type="text"]</code> , <code>:hover</code>
타입·의사 요소	0,0,0,1	<code>p</code> , <code>h1</code> , <code>::before</code>
전체 선택자	0,0,0,0	<code>*</code>

CSS 명시도(Specificity) 계산 가중치 비교 다이어그램

코드로 확인하기 — 명시도 충돌


```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>명시도 비교</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <p class="text" id="intro">이 문단에 여러 규칙이 충돌합니다.</p>
  <p class="text">클래스만 적용된 문단입니다.</p>
</body>
</html>

```

```

/* style.css */

/* 타입 선택자: 명시도 0,0,0,1 */
p {
  color: gray;
}

/* 클래스 선택자: 명시도 0,0,1,0 (타입보다 높음) */
.text {
  color: steelblue;
}

/* 아이디 선택자: 명시도 0,1,0,0 (클래스보다 높음) */
#intro {
  color: tomato;
}

/* 결과: #intro가 적용된 첫 번째 p는 tomato(빨간색) */
/* .text만 있는 두 번째 p는 steelblue(파란색) */

```

브라우저 렌더링 결과:

이 문단에 여러 규칙이 충돌합니다. ← 빨간색(tomato) - #intro 승리
 클래스만 적용된 문단입니다. ← 파란색(steelblue) - .text 적용

줄별 코드 설명

줄	코드	설명
3~5	<code>p { color: gray; }</code>	명시도 <code>0,0,0,1</code> . 모든 <code>p</code> 에 기본값으로 적용됩니다
8~10	<code>.text { color: steelblue; }</code>	명시도 <code>0,0,1,0</code> . 타입 선택자보다 높아 <code>steelblue</code> 가 <code>gray</code> 를 덮습니다
13~15	<code>#intro { color: tomato; }</code>	명시도 <code>0,1,0,0</code> . 클래스보다 높아 <code>tomato</code> 가 최종 적용됩니다

코드로 확인하기 — 선택자 복합 명시도

여러 선택자가 결합되면 점수가 합산됩니다.

```
/* 명시도 계산 예시 */
```

```
/* 0,0,0,1 (타입 1개) */
```

```
p { color: black; }
```

```
/* 0,0,1,1 (클래스 1 + 타입 1) */
```

```
p.text { color: navy; }
```

```
/* 0,1,1,1 (아이디 1 + 클래스 1 + 타입 1) */
```

```
#wrap p.text { color: tomato; }
```

```
/* 명시도가 같은 경우: 나중에 선언된 것이 우선 */
```

```
.text { color: steelblue; }
```

```
.text { color: orchid; } /* orchid가 적용됨 (동일 명시도, 후선언 우선) */
```

팁: 브라우저 개발자 도구(F12)의 Elements > Styles 패널에서 어떤 규칙이 적용되고 어떤 규칙이 취소선(명시도 패배)이 그어져 있는지 확인할 수 있습니다. 명시도 충돌을 디버깅할 때 가장 유용한 도구입니다.

상속(Inheritance)

일부 CSS 속성은 부모 요소의 값을 자식 요소가 자동으로 물려받습니다.

- 상속되는 속성: `color`, `font-family`, `font-size`, `line-height`, `text-align` 등 텍스트 관련 속성
- 상속되지 않는 속성: `width`, `height`, `margin`, `padding`, `border`, `background` 등 박스 모델 속성

```
/* 부모에 color를 지정하면 자식 텍스트에 자동 적용됨 */
body {
  color: #333;
  font-family: 'Noto Sans KR', sans-serif;
}

/* 모든 p가 body의 color: #333을 상속받음 (별도 선언 불필요) */
```

상속을 강제하거나 취소하고 싶을 때는 키워드를 사용합니다.

```
p {
  color: inherit; /* 부모 값 강제 상속 */
  font-size: initial; /* 브라우저 기본값으로 초기화 */
  border: unset; /* inherit 가능하면 inherit, 아니면 initial */
}
```

!important는 최후의 수단

`!important` 를 선언에 추가하면 모든 명시도 계산을 무시하고 최우선으로 적용됩니다.

```
/* 이 선언은 아이디 선택자도 이깁니다 */
p { color: green !important; }

/* 하지만 !important끼리 충돌하면 다시 명시도 계산으로 결정됩니다 */
```

주의: `!important` 는 캐스케이드를 깨뜨려 디버깅을 매우 어렵게 만드는 안티패턴입니다. 서드파티 CSS를 덮어써야 할 때 등 불가피한 경우에만 사용하고, 평소에는 명시

도 설계로 해결합니다.

코드로 확인하기 — 실전 스타일 시트 구조

좋은 CSS 파일은 명시도를 의도적으로 낮게 유지합니다.

```
/* =====  
style.css — 권장 구조 예시  
===== */  
  
/* 1. 전역 초기화 (낮은 명시도) */  
*,  
*::before,  
*::after {  
  box-sizing: border-box;  
}  
  
/* 2. 베이스 스타일 (타입 선택자) */  
body {  
  font-family: 'Noto Sans KR', sans-serif;  
  color: #333;  
  line-height: 1.6;  
}  
  
/* 3. 컴포넌트 스타일 (클래스 선택자 위주) */  
.btn {  
  display: inline-block;  
  padding: 8px 20px;  
  background-color: #1a73e8;  
  color: #fff;  
  border-radius: 4px;  
  text-decoration: none;  
  border: none;  
  cursor: pointer;  
}  
  
.btn:hover {  
  background-color: #1558b0;  
}  
  
/* 4. 변형(modifier) 클래스 */  
.btn--danger {  
  background-color: #d32f2f;  
}
```

줄별 코드 설명

줄	코드	설명
6~8	<code>*, *::before, *::after { box-s...</code>	모든 요소와 의사 요소에 <code>border-box</code> 를 적용합니다. 명시도가 0이어서 나중에 쉽게 재정의할 수 있습니다
11~15	<code>body { ... }</code>	타입 선택자로 전역 기본값을 지정합니다. 자식 요소에 color와 font-family가 상속됩니다
18~27	<code>.btn { ... }</code>	클래스 선택자로 재사용 가능한 버튼 컴포넌트를 정의합니다
29~31	<code>.btn:hover { ... }</code>	의사 클래스로 마우스 오버 시 스타일을 변경합니다 (ch_08에서 자세히 다룸)
34~36	<code>.btn--danger { ... }</code>	BEM 방식의 수정자(modifier) 클래스. 기본 <code>.btn</code> 의 색만 재정의합니다

비교 테이블 — 명시도 점수 예시

선택자	명시도	계산 근거
<code>*</code>	0,0,0,0	전체 선택자
<code>p</code>	0,0,0,1	타입 1개
<code>.btn</code>	0,0,1,0	클래스 1개
<code>p.text</code>	0,0,1,1	클래스 1 + 타입 1
<code>#header</code>	0,1,0,0	아이디 1개
<code>#header .nav li</code>	0,1,1,1	아이디 1 + 클래스 1 + 타입 1
<code>style=" ... "</code>	1,0,0,0	인라인 스타일

절 요약

- 캐스케이드는 중요도 → 명시도 → 선언 순서 우선순위 규칙입니다.

- 명시도는 인라인(1,0,0,0) > 아이디(0,1,0,0) > 클래스/속성/의사 클래스(0,0,1,0) > 타입(0,0,0,1) 순입니다.
 - `color` · `font-family` 등 텍스트 속성은 부모에서 자식으로 상속됩니다.
 - `!important` 는 캐스케이드를 무너뜨리므로 최후의 수단으로만 사용합니다.
 - 명시도 충돌은 개발자 도구(F12)의 Styles 패널로 진단합니다.
-

FAQ

Q1: 인라인 스타일과 외부 스타일시트 중 어느 쪽이 우선 적용되니까?

인라인 스타일이 우선됩니다. 인라인 스타일의 명시도는 `1,0,0,0` 으로 가장 높기 때문입니다. 이것이 인라인 스타일을 지양하는 이유 중 하나입니다. 외부 스타일시트에서 인라인 스타일을 덮어쓰려면 `!important` 를 써야 하는데, 이는 유지보수를 어렵게 만듭니다.

Q2: `padding` 과 `margin` 의 차이가 무엇입니까?

`padding` 은 콘텐츠와 테두리(border) 사이의 **안쪽** 여백입니다. 요소의 배경색이 패딩 영역까지 채워집니다. 반면 `margin` 은 테두리 바깥의 **외부** 여백으로, 다른 요소와의 간격을 만듭니다. 배경색은 마진 영역에 적용되지 않습니다.

Q3: `em` 과 `rem` 중 무엇을 써야 합니까?

용도에 따라 다릅니다. `rem` 은 루트 폰트 크기를 기준으로 하여 중첩 계층의 영향을 받지 않아 예측 가능합니다. 글자 크기(`font-size`)와 전반적인 레이아웃 크기에는 `rem` 을 권장합니다. `em` 은 특정 컴포넌트 내부에서 폰트 크기에 비례하여 여백을 맞출 때(예: 버튼 패딩) 유용합니다.

Q4: `class` 와 `id` 선택자를 언제 구분해 사용합니까?

`class` 는 여러 요소에 반복 적용할 수 있어 스타일 목적에 적합합니다. `id` 는 페이지 내 유일한 요소를 식별하는 용도이므로 페이지 내 앵커 링크, JavaScript `getElementById()` 등에 씁니다. CSS 스타일링에는 `id` 선택자보다 `class` 선택자를 권장합니다. 명시도가 낮아 재정의가 쉽기 때문입니다.

Q5: 스타일이 예상대로 적용되지 않을 때 어떻게 디버깅합니까?

브라우저 개발자 도구(F12 또는 Ctrl+Shift+I)를 열고, Elements 패널에서 해당 요소를 선택합니다. 오른쪽 Styles 탭에서 어떤 규칙이 적용되고 어떤 규칙이 취소선으로 표시되는지 확인합니다. 취소선 규칙은 명시도 경쟁에서 패배한 것입니다. Computed 탭에서 최종 적용 값도 확인할 수 있습니다.

장 요약

이번 장에서는 CSS의 핵심 기초를 다졌습니다. CSS를 HTML에 연결하는 세 가지 방법 중 외부 스타일시트가 관심사 분리와 재사용성 측면에서 가장 권장됩니다. 선택자는 타입·클래스·아이디와 결합자를 조합하여 원하는 요소를 정밀하게 지정합니다. 박스 모델은 content·padding·border·margin으로 구성되며, `box-sizing: border-box` 를 적용하면 크기 계산이 직관적으로 단순해집니다. 마지막으로 캐스케이드와 명시도 규칙은 스타일 충돌을 해결하는 핵심 메커니즘으로, 이를 이해하면 예측 가능한 CSS를 작성할 수 있습니다.

핵심 키워드

외부 스타일시트 , link 태그 , 타입 선택자 , 클래스 선택자 , 아이디 선택자 , 결합자 , 속성 선택자 , 박스 모델 , content/padding/border/margin , box-sizing , 마진 상쇄 , hex/rgb/hsl , px/em/rem , 명시도 , 캐스케이드 , 상속 , !important

실습 프로젝트: 스타일이 입혀진 프로필 카드

목표: 외부 스타일시트로 프로필 카드 컴포넌트를 디자인합니다. 박스 모델로 여백과 테두리를 다듬고, 클래스 선택자와 색상·단위를 활용하여 보기 좋은 카드를 완성합니다.

활용 개념: 외부 스타일시트, 선택자(클래스·타입·결합), 박스 모델, `box-sizing: border-box` , 색상과 단위, 명시도

다음 장 미리보기

6장에서는 CSS 레이아웃의 핵심인 **Flexbox**와 **Grid**를 배웁니다. 이 장에서 배운 박스 모델과 display 속성을 기반으로, 요소를 원하는 위치에 자유롭게 배치하는 방법을 익히게 됩니다.

✂ 실습 프로젝트 (Hands-on Project)

5장 실습 프로젝트: 스타일이 입혀진 프로필 카드

이 장에서 배운 CSS 기초 개념을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 프로필 카드 만들기

시나리오

온라인 커뮤니티에서 사용자 프로필을 보여주는 카드 컴포넌트를 만들어야 합니다. 디자이너로부터 "이름, 직업, 소개 문구, 기술 태그가 들어간 흰 배경의 둥근 카드"를 만들어 달라는 요청을 받았습니다. 외부 스타일시트로 스타일을 분리하여 나중에 디자인만 쉽게 바꿀 수 있도록 구현합니다.

학습 목표

- 외부 스타일시트 연결과 관심사 분리를 실습합니다
- 타입-클래스 선택자와 자손 결합자로 카드 내부를 스타일링합니다
- box-sizing: border-box로 패딩이 있어도 너비를 정확히 제어합니다
- hex/rgb 색상과 px/rem 단위를 직접 사용합니다

진행 방법

1. `project_starter.html` 과 `project_starter_style.css` 파일을 열어 TODO 주석을 찾습니다
2. 각 TODO를 이 장에서 배운 CSS 개념으로 완성합니다
3. 브라우저에서 열어 프로필 카드가 올바르게 보이는지 확인합니다

실행 방법

브라우저에서 `project_starter.html`을 열거나 VS Code Live Server 사용

완성 예시

`project_complete.html` + `project_complete_style.css` 파일을 참고하세요.

중급: 다색 프로필 카드 그리드 페이지

시나리오

초급에서 만든 프로필 카드를 발전시켜 여러 장의 카드를 격자로 배치하는 페이지를 만듭니다. 각 카드마다 다른 강조 색상을 클래스 수정자(modifier)로 적용하고, 명시도 설계를 활용하여 베이스 카드 스타일 위에 변형을 쌓는 패턴을 실습합니다. 속성 선택자로 외부 링크를 내부 링크와 시각적으로 구분합니다.

학습 목표

- 클래스 수정자 패턴으로 낮은 명시도를 유지하며 변형을 만듭니다
- 자손·자식 결합자와 속성 선택자를 조합합니다
- 상속을 활용하여 중복 CSS를 줄입니다
- hsl() 색상으로 일관된 색상 계열을 만듭니다

진행 방법

1. `project_advanced_starter.html` 과 `project_advanced_starter_style.css` 파일을 열고 TODO를 찾습니다
2. 초급에서 연습한 내용을 바탕으로 복잡한 선택자 조합과 명시도 설계를 구현합니다
3. 완성 후 브라우저에서 여러 색상 카드가 배치되는지 확인합니다

실행 방법

브라우저에서 `project_advanced_starter.html`을 열거나 VS Code Live Server 사용

완성 예시

`project_advanced_complete.html` + `project_advanced_complete_style.css` 파일을 참고하세요.

확장 아이디어

- 카드에 사용자 아바타 이미지(img + alt)를 추가해 보세요
- `:hover` 의사 클래스로 마우스를 올렸을 때 카드 효과를 추가해 보세요 (8장)
- 미디어 쿼리로 화면이 좁을 때 카드가 한 줄로 바뀌도록 해보세요 (7장)

▶  project_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_starter_style.css (스타터 코드 - 직접 완성해보세요)

▶  project_complete.html (완성 코드)

▶  project_complete_style.css (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_starter_style.css (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)

▶  project_advanced_complete_style.css (완성 코드)